

INTERNAL TECHNICAL DOCUMENT

E2EE Chat

Complete platform specification

Every implemented feature, algorithm, event, and limitation in this repository — cryptography, fingerprints, presence, signed messages, groups, View once media, WebRTC calls, notifications, and the relay server.

This document is generated from the actual code paths in `server.js` and `public/app.js`. It is not a marketing overview.

Package: `e2ee` · version 1.0.0

Runtime: Node.js + browser Web Crypto + WebRTC

Relay: Express 5.2 + Socket.IO 4.8 on port 3000

Date of this snapshot: 13 September 2026

Contents

1. Purpose and how to read this document
2. System architecture
3. Technology stack
4. Trust model and what the server can see
5. Cryptographic building blocks
6. Session bootstrap and key lifecycle
7. Safety numbers (fingerprints)
8. Display names, avatars, and the peer directory
9. Presence, online status, and disconnect
10. Direct (1:1) end-to-end messaging
11. Signed messages (authentication)
12. Delivery and read receipts
13. Typing indicators
14. Emoji reactions
15. Groups and sender keys
16. Photos, video, attachments, and View once
17. Voice and video calls (WebRTC)
18. Browser notifications and in-app toasts
19. Client UI systems
20. Relay server internals
21. Complete Socket.IO event catalog
22. In-memory client and server state
23. Constants, limits, and identifiers
24. Encoding and serialization
25. Security properties and honest limitations
26. Repository map
27. Worked examples (message + call)
28. Glossary

1. Purpose and how to read this document

This specification describes **this** E2EE Chat platform as implemented. Every algorithm name, curve, IV length, Socket.IO event, timeout, and UI behavior below is taken from `server.js` and `public/app.js`. If a popular messenger does something differently (Signal protocol, sealed sender, TURN, persistent accounts), that difference is called out rather than implied.

The product is a browser chat app: two or more tabs or devices connect to a Node relay. The relay forwards ciphertext, public keys, presence, group membership, call signaling, and display names. It never receives message plaintext, attachment bytes in the clear, or long-term private keys.

Private keys live only in the tab that generated them. Reloading the page destroys identity. That is an implementation fact, not an accident of this write-up.

2. System architecture

2.1 Components

Three moving parts:

- **Browser client** (`public/index.html`, `styles.css`, `app.js`) owns keys, encryption, signatures, media, WebRTC, and UI.
- **Relay server** (`server.js`) is an Express static file host plus a Socket.IO message switch.
- **Peer browsers** are other tabs or devices running the same client. There is no account database, no login, and no offline store.

2.2 Data plane vs control plane

Chat **content** (text and file bytes) is encrypted with AES-GCM before it touches the socket. Call **media** (microphone and camera) never goes through Node at all: after SDP/ICE signaling, audio and video are peer-to-peer DTLS-SRTP inside the browsers. Everything else — who is online, display names, public keys, group membership, typing flags, receipt IDs, reaction emoji, View-once-opened acks, and WebRTC SDP/ICE — is plaintext metadata on the relay.

2.3 Connection path

The browser loads `http://localhost:3000/`. Express serves the static files with `Cache-Control: no-store` for HTML, CSS, and JS. The page then opens a Socket.IO connection to the same origin. Socket.IO 4 uses Engine.IO underneath (WebSocket when possible, HTTP long-polling as fallback). The server listens on **port 3000** on all interfaces.

2.4 Identity in one sentence

A user is the Socket.IO socket `.id` of the current tab, plus two ephemeral P-256 key pairs generated in that tab, plus a display name string. There is no password, no user ID, and no device list.

3. Technology stack

Layer	Technology	Where / why
Runtime (server)	Node.js (CommonJS)	Loads <code>server.js</code> . Start command: <code>npm start → node server.js</code> .
HTTP	Express ^5.2.1	Static files from <code>public/</code> . No REST API for chat.
Realtime	Socket.IO ^4.8.3	All chat, presence, groups, receipts, reactions, View-once acks, and call signaling.
Crypto	Web Crypto API (<code>crypto.subtle</code>)	ECDH, ECDSA, AES-GCM, SHA-256, CSPRNG. Runs only in the browser.
Randomness	<code>crypto.getRandomValues</code>	12-byte AES-GCM IVs.
Calls	WebRTC (<code>RTCPeerConnection</code> , <code>getUserMedia</code>)	Voice and video. DTLS-SRTP for media. SDP/ICE over Socket.IO.
NAT traversal	STUN only: <code>stun:stun.l.google.com:19302</code>	No TURN server is configured. Some NATs will fail to connect.
Notifications	Browser Notification API	Optional; requires permission. Not push/FCM.
Audio (ringtone)	Web Audio API <code>OscillatorNode</code>	680 Hz beep, 180 ms, every 1.1 s while ringing.

Layer	Technology	Where / why
Persistence	localStorage only	Keys: <code>e2ee.displayName</code> , <code>e2ee.theme</code> . No IndexedDB. No key storage.
UI	Vanilla HTML/CSS/JS	No React/Vue. CSS variables for light/dark.
Media preview	<code>URL.createObjectURL / revokeObjectURL</code>	Decrypted bytes become blob URLs in the tab.

Table 1. Stack as declared in `package.json` and as used in the browser.

4. Trust model and what the server can see

4.1 What the client trusts

- The browser's Web Crypto implementation (origin-bound, not polyfilled).
- That the JavaScript served from this origin was not replaced (there is no subresource integrity on the local files).
- That the peer's ECDH and ECDSA public keys belong to the human you think they do — this is what fingerprints are for.
- That Socket.IO delivers events to the `socket.id` you addressed (the relay can lie; signatures detect tampering of content, not of routing).

4.2 What the relay is allowed to see (and does)

- Every socket id, connect and disconnect time.
- Display names (plaintext).
- Exported public keys (JWK): ECDH encryption key and ECDSA signing key.
- Who sends to whom (to / from).
- Ciphertext and IV (base64). The relay cannot decrypt them without private keys.
- Attachment metadata that travels beside ciphertext: filename, MIME type, size, kind, View-once flag.
- Group names, member lists, and room joins/leaves.
- Typing start/stop, reaction emoji, delivery/read message IDs, View-once-opened acks.
- WebRTC SDP offers/answers and ICE candidates (which include IP addresses).

4.3 What the relay must not see (and does not, if the client is honest)

- ECDH or ECDSA private keys.
- AES-GCM keys (pairwise or group sender keys).
- Message plaintext.
- Raw photo/video bytes (only AES-GCM ciphertext).
- Microphone or camera samples (those stay on the WebRTC peer connection).

A modified client can send plaintext. The protocol does not force other honest clients to accept unsigned or unverifiable payloads: 1:1 and group handlers verify ECDSA before showing content, and failed verification is shown as an error.

5. Cryptographic building blocks

All of the following are Web Crypto algorithm identifiers passed verbatim to `crypto.subtle`. There is no custom cipher, no Signal Double Ratchet, no X3DH, no prekeys, and no deniable encryption.

Job	Algorithm	Parameters	Extractable / usages
Key agreement (1:1 and wrapping group sender keys)	ECDH	namedCurve P-256 (NIST secp256r1)	Keys generated extractable. Private usage: <code>deriveKey</code> . Peer public imported with empty usages.
Message and file encryption	AES-GCM	Key length 256 bits. IV 12 bytes (96-bit nonce) from CSPRNG. Web Crypto appends a 128-bit GCM tag to the ciphertext.	Pairwise derived keys: extractable false . Group sender keys: generated extractable so they can be exported raw and wrapped.
Signatures	ECDSA	namedCurve P-256 . Hash SHA-256 .	Generated extractable. Usages <code>sign</code> and <code>verify</code> . Peer public imported with <code>verify</code> only.
Fingerprints and (implicit) integrity of exported public keys	SHA-256	Digest of the raw ECDH public key bytes.	Hash only; not used as a MAC on messages (ECDSA covers that).
IV / key generation	CSPRNG	<code>crypto.getRandomValues</code> for IVs; Web Crypto for keys.	IVs are never reused on purpose: a new 12-byte IV is created for every encrypt call.

Table 2. Algorithms actually invoked. P-256 is NIST P-256, not Curve25519.

5.1 Why this combination

ECDH on P-256 is available in every modern browser without extra libraries. Each pair of users derives a shared AES-256-GCM key from (my private ECDH key, their public ECDH key). Because ECDH is symmetric in the two public points, Alice→Bob and Bob→Alice produce the same AES key. AES-GCM then provides confidentiality and ciphertext authenticity for that pair. ECDSA is a second key pair so that a man-in-the-middle who only swapped encryption keys still cannot forge a signature unless they also swapped signing keys — which fingerprints are meant to catch.

5.2 What this is not

- **Not Signal / Double Ratchet.** There is no per-message chain key, no skipped-message keys, and no future secrecy beyond 'close the tab'. Compromise of a private ECDH key decrypts all 1:1 traffic that used the corresponding pairwise AES key for that session.
- **Not X25519 / Ed25519.** The curve is NIST P-256.
- **Not password-authenticated.** No PAKE, no recovery codes.
- **Not sealed sender.** The relay always sees from as the socket id.

6. Session bootstrap and key lifecycle

6.1 On page load (`init()`)

- Apply theme from `?theme=`, else `localStorage.e2ee.theme`, else `prefers-color-scheme`.
- Read display name from `localStorage.e2ee.displayName` if present.
- `crypto.subtle.generateKey` for ECDH P-256 (extractable, usage `deriveKey`).

- Export the ECDH public key as JWK → `myPublicKeyJwk`.
- `generateKey` for ECDSA P-256 (extractable, sign+verify).
- Export the ECDSA public key as JWK → `mySigningPublicKeyJwk`.
- If no saved display name, generate a dummy name from the ECDH JWK coordinates (see §8).
- Compute the local fingerprint from the ECDH public key (see §7).
- If the socket is already connected, emit `register-key`.

6.2 Registration payload

Event `register-key` body:

```
{
  encryptKey: <JWK of ECDH public key>,
  signKey:    <JWK of ECDSA public key>,
  displayName: <string, max 32 chars after sanitize>
}
```

The server stores that triple under `publicKeys[socket.id]` and broadcasts `peer-list` to every connected client. A later `update-name` changes only the display name and broadcasts again. Registration is rejected if either public key is missing.

6.3 What is never persisted

Private keys, derived AES keys, group sender keys, message history, receipts, and call state exist only in RAM in that tab. Closing or reloading the tab is a full identity reset. The next load is a new person as far as the network is concerned (new socket id, new keys). Only the display-name string and theme survive, via `localStorage`.

6.4 Importing peers

On `peer-list`, the client skips its own socket id. For every other entry with both keys, it `importKeys` the ECDH JWK (usages `[]`) and the ECDSA JWK (usages `['verify']`), stores them in `peerKeys` and `peerSigningKeys`, and records `peerProfiles[id].displayName`. Failed imports are logged and that peer is omitted from the visible list.

7. Safety numbers (fingerprints)

Fingerprints let two humans confirm they have the same ECDH public key, out of band (read aloud, compare screens). They are computed independently on each device from the local copy of a public key. The server is not involved.

7.1 Algorithm

- Export the ECDH public key with `exportKey('raw')`. For P-256 this is the uncompressed EC point (0x04 || X || Y), 65 bytes.
- SHA-256 that byte string → 32-byte digest.
- **Word form:** four words from `WORD_LIST`, joined by hyphens. Indexes are `hash[0]`, `hash[3]`, `hash[6]`, `hash[9]`, each modulo the list length.
- **Hex form:** the first 8 bytes of the digest, each printed as two lowercase hex digits (16 hex characters, no separators).

Example shape: `echo-gravel-xray-quoll` plus hex `1e2fe7a07d634a3d`. The UI shows words in a large monospace line and hex underneath.

7.2 Where they appear

Your own fingerprint is computed once after key generation and shown in the Verify identity dialog. The selected 1:1 peer's fingerprint is computed when you open that chat (`selectPeer`) from `peerKeys[id]`. Groups do not show a pairwise fingerprint; the Verify button is hidden in group chats. Users are instructed to read the words aloud.

7.3 What a match proves

Matching words mean both devices hashed the same raw ECDH public key. That binds the encryption key. It does **not** by itself bind the ECDSA signing key (a separate JWK). In this app both keys are sent together on `register-key` and both are imported from the same peer-list row, so a relay that swaps only one of them would still be visible as a fingerprint mismatch on the encryption key, or as signature failures if only the signing key were swapped.

Fingerprints are not SAS from a PAKE and not QR codes. There is no 'mark as verified' bit stored; verification is a human procedure each session.

8. Display names, avatars, and the peer directory

8.1 Design rule

Socket IDs are implementation identifiers. The UI never shows them. Chats, headers, notifications, call screens, and group member lists use display names. The server still routes by socket id.

8.2 Dummy name generation

If `localStorage` has no name after keys exist, the client builds one from the ECDH public JWK:

- Seed string: `jwk.x + ':' + jwk.y` (the JWK coordinates).
- 32-bit FNV-1a: offset basis 2166136261, prime 16777619, XOR then multiply for each character code.
- Adjective = `NAME_ADJECTIVES[abs(hash) % 24]`.
- Noun = `NAME_NOUNS[abs(hash >>> 8) % 24]`.
- Result like `Calm Heron` or `Brave Grove`.

The name is written to `localStorage` and included in `register-key`. The user can edit the sidebar field (max 32 characters). Input is debounced 400 ms and also sent on change via `update-name`.

8.3 Server-side sanitize

`sanitizeDisplayName` requires a string, collapses whitespace, trims, and slices to 32 characters. Empty names become Anonymous on the server. Group names use the same sanitizer.

8.4 Avatars

Initials: first letter of the first word plus first letter of the second word, uppercased (one letter if a single word). Color: 32-bit rolling hash `hash = hash * 31 + charCode` of a seed (socket id for people, group id for groups), modulo 12 colors. This is cosmetic, not a security property.

8.5 Peer list rendering

The Chats list is `visiblePeerIds` — everyone in the last peer-list who is not you and whose keys imported. Each row shows avatar, display name, a green presence dot, preview text (last message snippet, or 'typing...', or 'Tap to chat'), and an unread badge. There is no 'Peers' heading exposing raw IDs.

9. Presence, online status, and disconnect

9.1 Online means 'registered in this process'

A peer is online if their socket is connected **and** they have successfully emitted `register-key`. The server object `publicKeys` is the source of truth. `peer-list` is broadcast on `register`, on name change, and on disconnect.

9.2 Client handling of peer-list

The client rebuilds `visiblePeerIds`. Keys for IDs no longer in the list are deleted, and those IDs are removed from `typingPeers`. If you are in a 1:1 chat with someone who disappeared, the client toasts that they went offline and returns to the welcome screen. If a call is up with a vanished peer, the call is torn down locally without sending hangup (the socket is already gone).

9.3 Disconnect on the server

Socket.IO `disconnect`: delete `publicKeys[id]`, remove the id from every group (deleting empty groups), broadcast `peer-list`, and broadcast `group-list` if membership changed. There is no 'last seen' timestamp and no offline inbox.

9.4 Reconnect

The client shows a top banner 'Reconnecting to the relay...'. On connect it hides the banner, re-registers keys, and requests groups. Because keys are still in RAM if the **socket** dropped but the **tab** did not, the same key pair is re-registered — but the socket id is new, so everyone else sees a new peer. If the tab itself reloaded, keys are new too.

9.5 Header presence copy

1:1 subtitle: online · end-to-end encrypted, or typing... while a typing event is active. Sidebar summary: N online. Group subtitle: member count and display names (you + others).

10. Direct (1:1) end-to-end messaging

10.1 Shared AES key

For peer P, the client calls `deriveKey` with algorithm ECDH, public = P's imported ECDH key, base key = my ECDH private key, derived key algorithm AES-GCM 256, extractable false, usages encrypt+decrypt. This is done on every send and every receive (no long-lived key cache). The operation is deterministic: both sides get the same AES key.

10.2 Sending text

- Read the composer, trim. Empty text is ignored.
- UTF-8 encode with `TextEncoder`.
- New 12-byte IV. AES-GCM encrypt. Base64 the IV and ciphertext (chunked; see §24).
- Build the signature payload (see §11) and ECDSA-sign it.
- Allocate `msgId` = `base36(timestamp)` + 4 random base36 chars.
- Emit send-message with `to`, `msgType` 'text', ciphertext, iv, both public JWKs, signature, `msgId`.
- Stop typing notifications. Append a local 'sent' bubble with tick 'sent'. Store the message under `messageStates[msgId]` for later ticks.

The plaintext is **not** on the wire. The signature is over the plaintext JSON, but only the signature bytes travel with the ciphertext.

10.3 Receiving text

- Immediately emit `msg-delivered` to the sender (the ciphertext arrived; decryption has not run yet).
- Clear typing state for that sender.
- Import sender keys if needed (messages also carry `senderPublicKey` / `senderSigningKey` so a race with `peer-list` still decrypts).
- Derive the same AES key. Decrypt. UTF-8 decode.
- Verify ECDSA over the reconstructed text payload.
- Append into conversation `peer:<from>`. Show a signed/unverified badge.
- Emit `msg-read` after successful handling (including after a failed decrypt path? only on the success path after append).
- Maybe show a desktop notification (§18).

10.4 Conversation isolation

Messages are stored per conversation key `peer:<socketId>` or `group:<groupId>`. Opening a chat renders only that array. If the conversation is not open, unread increments and a badge appears. Switching chats saves the composer draft in a Map keyed by conversation id and restores it on return. History is RAM-only.

10.5 Composer

Enter sends; Shift+Enter is a newline. The textarea autosizes up to 120 px. The composer is disabled until a peer or group is selected. Stick-to-bottom scrolling uses a 96 px slack: if you scrolled up, new messages do not yank the view.

11. Signed messages (authentication)

11.1 Why a second key

ECDH+AES-GCM authenticates the **ciphertext** under the shared key. Anyone who can derive that key (the two parties, or an attacker who swapped both ECDH keys) can produce valid GCM tags. ECDSA is an origin signature under a key that is not used for encryption, so recipients can show a 'signed' badge meaning: this payload matches the signing public key we associated with that peer.

11.2 Text payload (canonical JSON)

```
JSON.stringify({
  msgType: "text",
  text: <plaintext string>
})
```

That exact UTF-8 string is what `subtle.sign({name:'ECDSA', hash:'SHA-256'}, privateSignKey, bytes)` signs. Verification uses the same JSON on the receiver after decrypt. Field order is the insertion order above; do not pretty-print or reorder keys.

11.3 Attachment payload

```
JSON.stringify({
  msgType: "attachment",
```

```

    name, mimeType, size, kind,
    viewOnce: Boolean(viewOnce),
    ciphertext, // base64
    iv          // base64
  })

```

Signing the ciphertext (not the raw file) means the signature is over what actually went on the wire plus the metadata, including the View-once flag. A relay that flips `viewOnce` or swaps the ciphertext will fail verification.

11.4 Group payloads

Sender-key wrap:

```

JSON.stringify({
  kind: "group-sender-key",
  groupId, keyId, recipientId,
  ciphertext, iv
})

```

Group message:

```

JSON.stringify({
  kind: "group-message",
  groupId, keyId, msgId, msgType,
  ciphertext, iv,
  attachmentMeta // object or null
})

```

11.5 UI

A small badge on every non-system bubble: green 'signed' if `verify` returned true, red 'unverified' otherwise. Failed group signatures are rejected with a system error line and the body is not shown.

Display names are not signed. A relay can rename a peer in peer-list. Fingerprints still bind keys; they do not bind the label 'Calm Heron'.

12. Delivery and read receipts

This is WhatsApp-style ticks, implemented as two relayed events. They are **not** encrypted and not signed.

Tick	Meaning in this app	How it is set
✓ (gray, class tick-sent)	This tab emitted send-message (or the local echo of a group send).	Created with the outgoing bubble. Title 'sent'.
✓✓ (gray, tick-delivered)	The recipient's client received the socket event and emitted msg-delivered. Decryption may not have succeeded yet.	On msg-delivered, look up messageStates[msgId] and updateTick('delivered').
✓✓ (blue #53bdeb, tick-read)	The recipient finished the receive handler success path and emitted msg-read.	On msg-read, updateTick('read').

Table 3. Receipts. Group sends do not get ticks (no per-member ack protocol).

Both events are `relayToPeer: { to, msgId }`. The server adds `from`. A missing `to`, a non-string `to`, or sending to self is dropped. There is no retry if the original sender already disconnected.

13. Typing indicators

1:1 only. While the composer has non-empty text and a peer is selected, the client emits `typing-start` once, then resets a 1200 ms timer on every input. When the timer fires, or the field is cleared, or the chat is left, or the tab unloads,

it emits typing-stop.

Receivers add/remove the sender in `typingPeers`, refresh the sidebar ('typing...' + blue dot), and if that chat is open, set the header subtitle to 'typing...' and the line under the thread to '<name> is typing...'. Incoming messages also clear typing for that sender. Groups have no typing events.

14. Emoji reactions

Quick set, hard-coded in the client as the four Unicode emoji heart, thumbs-up, face-with-tears-of-joy, and fire (the JS array is those four characters). The picker is shown on **received 1:1** messages only (hover on fine pointers; tap toggles `.is-picker-open` on touch). Clicking an emoji toggles it: second click on the same emoji removes it. Only one local reaction is stored at a time; changing emoji sends remove for the old one then add for the new.

Wire: `message-reaction { to, msgId, emoji, action }` where action is 'add' or 'remove'. The original sender looks up `sent:<msgId>` in `messageRegistry` and sets `remoteReaction`. Chips render under the bubble ('you' vs the peer's display name). Reactions are not encrypted, not signed, and not implemented for groups.

15. Groups and sender keys

15.1 Room state on the server

Groups are in-memory: `id → { name, members: Set(socket.id) }`. Create emits `create-group { name }`; the server makes `grp_ + Date.now().toString(36) + 4` random chars, adds the creator, `socket.join(groupId)`, broadcasts `group-list`, and emits `group-selected` to the creator. Join/leave add or remove the socket from the Set and the Socket.IO room. Empty groups are deleted. Group-list rows include id, name, members array, `memberCount`.

15.2 Why sender keys

A naive group send would ECDH-wrap the message once per member (fan-out N encrypts). This app instead gives **each member their own AES-256-GCM sender key**. A member encrypts group traffic once with their sender key; everyone else decrypts with the copy they were given. Distribution of that key is pairwise: wrapped to each other member with the 1:1 ECDH AES key, then signed.

15.3 Local sender-key state (per group)

```
{
  mySenderKey,           // CryptoKey AES-GCM 256
  mySenderId,           // generateMsgId()
  remoteSenderKeys,     // { [socketId]: { keyId, key } }
  pendingMessages,      // ciphertext waiting for the matching keyId
  pendingDistributions, // member ids already wrapped this generation
  memberSnapshot         // sorted member ids joined by ':'
}
```

15.4 Rotation

On every group-list and after peer-list, if the member snapshot string changed, the client **resets** that group's sender-key state (drops my key, remote keys, and pending queues) and generates a fresh sender key. That is the entire membership ratchet: new members must not read old sender keys; leaving members should not receive new ones. There is no delayed-leave window.

15.5 Distribution

For each other member with a known ECDH key who is not already in `pendingDistributions`: export sender key as 32 raw bytes, ECDH-derive the pairwise AES key, AES-GCM encrypt those 32 bytes, sign the wrap payload, push {

to, ciphertext, iv, signature }. One `distribute-group-sender-key` event carries the array plus the sender's public encryption and signing JWKs. The server forwards each wrap only if both parties are still members and required fields exist.

15.6 Receiving a wrap

Verify ECDSA; reject with a system error if it fails. Decrypt the 32 bytes with the pairwise key, `importKey('raw', ... AES-GCM 256)`, store under `remoteSenderKeys[from]`, then drain `pendingMessages` for that sender.

15.7 Sending and receiving group messages

Send: ensure local sender key, AES-GCM encrypt with **that** key (not pairwise), sign the group-message payload, emit `send-group-message`. The server checks membership and does `socket.to(groupId).emit` (everyone in the room except the sender). Receive: if we do not yet have `remoteSenderKeys[from].keyId === keyId`, queue the packet. Otherwise verify, decrypt, and render. Attachments use the same key with `encryptBytes`.

Calls are 1:1 only. Group reactions, group typing, and group receipts are not implemented. The server can see group membership and ciphertext sizes.

16. Photos, video, attachments, and View once

16.1 What can be sent

The file picker accepts `image/*`, `video/*`. Drag-and-drop onto the conversation and clipboard paste of a file also open the media composer. Non-image/non-video files are rejected with a toast. Empty files are rejected. Client size limit: **12 MiB** (`ATTACHMENT_SIZE_LIMIT`). Socket.IO `maxHttpBufferSize` is **20 MiB** to leave room for JSON/base64 expansion (~4/3) plus envelope fields.

16.2 Composer

Choosing a file does not send immediately. A modal previews the image or video, shows name and size, and offers a **View once** switch (default off). Cancel revokes the preview object URL. Send encrypts, then closes the modal.

16.3 Encryption

The file is read as `Uint8Array` via `arrayBuffer()`. 1:1 uses the pairwise AES-GCM key; groups use the sender key. Same 12-byte IV + AES-GCM as text. Metadata traveling with the ciphertext:

```
attachmentMeta = {
  name, mimeType, size,
  kind: "image" | "video" | "file",
  viewOnce: true | false
}
```

Kind is derived from the MIME prefix `image/` or `video/`. The signature covers metadata + ciphertext + iv (§11.3).

16.4 Normal media (viewOnce false)

After decrypt, the client creates a blob URL and renders an `<img>` or `<video controls playsinline>`. Clicking a normal image opens a lightbox. The sender keeps a local blob URL of the original bytes (not re-downloaded).

16.5 View once — what it is

A UX constraint inspired by WhatsApp view-once, **not** a cryptographic self-destruct that the recipient's OS cannot bypass. The ciphertext is ordinary AES-GCM. The flag tells honest clients to withhold a persistent preview.

Recipient, unopened

Decrypted bytes stay in RAM on the message object (`attachment.bytes`). `downloadUrl` is null. The bubble is a tile: 'View once photo' or 'View once video' — 'Tap to open. It disappears after viewing.' No thumbnail.

Recipient, opening

Click creates a blob URL, opens a fullscreen viewer. Videos autoplay, `controlsList='nodownload noemotep playback'`, picture-in-picture disabled; ended → consume. Closing the viewer (X, overlay click, Escape) always consumes.

Consume

Revoke the blob URL. Set `opened=true`. Null out bytes and `downloadUrl`. Re-render the tile as 'Opened'. If the conversation key starts with `peer:`, emit `view-once-opened { to: sender, msgId }` so the sender can mark `openedByPeer`. Group view-once does not notify the whole room; consumption is local.

Sender

Keeps a normal preview of what they sent, labeled View once. When the 1:1 peer opens it, the sender's tile can show Opened while still displaying their own copy.

16.6 What View once does not do

- Does not stop screenshots, screen recording, or a modified client saving the bytes.
- Does not make the relay delete ciphertext (the relay never stored it beyond in-flight socket buffers).
- Does not survive reload: RAM is gone, so an unopened view-once is simply lost.
- Does not encrypt with a special one-time key; it is the same AES-GCM session key.

16.7 Historical bug that was fixed

An earlier path used `btoa(String.fromCharCode(...bytes))` on the whole file. Spreading a multi-megabyte `Uint8Array` overflows the JS call stack ('Maximum call stack size exceeded'). Encoding now chunks at 0x8000 bytes (§24).

17. Voice and video calls (WebRTC)

17.1 Scope

1:1 only. Buttons in the chat header: phone (audio) and camera (video). Groups hide both. One call session per tab. A second offer while busy is answered with `type: 'busy'`.

17.2 Media capture

`getUserMedia` constraints:

```

audio: true
video: false                // voice call
video: {                    // video call
  facingMode: "user",
  width: { ideal: 1280 },
  height: { ideal: 720 }
}

```

Tracks are added to `RTCPeerConnection` with `addTrack`. Local video is shown muted (`#local-video`) so you do not hear yourself. Remote stream is attached in `pc.ontrack`. Audio-only calls add class `audio-only`, hide both

video elements, and show a large avatar instead. The camera button is hidden on voice calls.

17.3 Peer connection

```
new RTCPeerConnection({
  iceServers: [{ urls: "stun:stun.l.google.com:19302" }]
})
```

No TURN, no multiple STUN, no iceTransportPolicy. Browsers will gather host, srflx (via Google STUN), and possibly existing relay candidates from the OS, but this app never provisions a TURN URI. Symmetric NAT / firewall pairs may fail. Localhost and many home NATs succeed.

17.4 Signaling (all over Socket.IO call-signal)

Client emits { to, data }. Server relays { from, data } if to is a different string. data.type is the discriminator:

data.type	Who sends it	Payload	Receiver action
offer	Caller, after setLocalDescription(offer)	sdp: RTCSessionDescription, callType: 'audio' 'video'	If idle: store remoteOffer, show incoming UI, start ringtone, 40 s timeout. If busy: reply busy.
answer	Callee, after accept + setLocalDescription(answer)	sdp: RTCSessionDescription	Caller setRemoteDescription, flush queued ICE.
ice	Either, on onIceCandidate	candidate: RTCIceCandidate	If remote description not set yet, push on pendingIce[]; else addIceCandidate.
hangup	Either, user ends or timeout 'No answer'	(none required)	cleanupCall(); toast 'Call ended'.
reject	Callee Decline, or 40 s ring timeout, or getUserMedia failure on accept	(none)	Caller cleanup; toast 'Call declined'.
busy	Callee already in callSession	(none)	Caller cleanup; toast 'Busy'.

Table 4. Call signaling. SDP and ICE are not encrypted at the application layer.

17.5 Offer/answer sequence (caller)

- getUserMedia → create RTCPeerConnection → addTrack each track.
- createOffer → setLocalDescription(offer) (this starts ICE gathering).
- signal offer + callType. Show in-call UI with status 'Calling...'.
 - 40 s timer: if no answer, hangup with reason 'No answer'.
- On answer: setRemoteDescription, flushIce.
- onconnectionstatechange 'connected' → start MM:SS timer, set status to 'Voice call' or 'Video call'.
- 'failed' or 'closed' → local end without hangup notify.

17.6 Callee accept

Incoming UI shows display name and 'Incoming voice/video call'. Accept is a user gesture (required for getUserMedia). Flow: getUserMedia → new PC → addTrack → setRemoteDescription(stored offer) → flush queued ICE → createAnswer → setLocalDescription → signal answer → show in-call 'Connecting'.

17.7 In-call controls

- **Mute:** toggles enabled on all local audio tracks. Button shows Mute / Unmute.
- **Camera:** toggles local video tracks. Off looks like a black/empty remote tile depending on the browser.
- **Hang up:** stop all local tracks, `pc.close()`, clear videos, signal hangup, hide overlays.
- `beforeunload` also sends hangup if a session exists.

17.8 Media encryption (browser, not our code)

WebRTC encrypts media with **DTLS-SRTP** (DTLS 1.2 handshake, then SRTP keys). That is inside Chromium/Firefox/WebKit. This codebase does not implement SRTP, does not pin DTLS certificates against the chat ECDH fingerprint, and does not use insertable streams / E2EE inserts. Identity for the call is 'the socket we signaled', plus whatever ICE path the browsers negotiated. An attacker who can MITM signaling and the media path could sit in the middle; an attacker who can only see the Node relay sees SDP/ICE but not the SRTP packets if the P2P path does not go through them.

17.9 Ringtone

Web Audio oscillator 680 Hz, gain 0.05, 180 ms pulse, interval 1100 ms. Autoplay may fail until a gesture; the incoming modal is still shown. `stopRingtone` closes the `AudioContext`.

17.10 Timeouts and glare

Incoming ring: 40 s then reject. Outgoing: 40 s then hangup 'No answer'. There is no glare-resolution (no 'lower socket id wins'). Simultaneous calls → one side is busy. No group calls, no screen share, no renegotiation mid-call (you cannot upgrade voice to video without a new call).

18. Browser notifications and in-app toasts

18.1 Permission

The bell button calls `Notification.requestPermission()`. If the API is missing, the button is disabled. If permission is granted or denied, the button is disabled and the title explains the state. There is no server-side push, no service worker, and no notification after the tab is fully closed.

18.2 When a notification fires

`maybeShowNotification(title, body, tag, conversationKey)` returns immediately unless permission is granted. It also suppresses the popup if the document is visible, focused, **and** the active conversation is the one the event belongs to. Clicking a notification focuses the window.

Sources: 1:1 text, 1:1 attachment, group text, group attachment, reaction add on one of your sent messages.

18.3 Toasts

Ephemeral in-app messages (4.2 s) for errors and call outcomes: send failures, media too large, already in a call, call declined/busy/ended, peer offline, `getUserMedia` errors. They are not encrypted events; they are local UI.

19. Client UI systems

19.1 Layout

Desktop: CSS grid, 360 px sidebar + chat pane. Mobile (max-width 860 px): sidebar fills the screen; opening a chat hides the sidebar and shows the chat pane full screen with a back button. Welcome pane shows until a conversation is selected.

19.2 Theme

`document.documentElement.dataset.theme` is `light` or `dark`. CSS custom properties switch surfaces, bubbles, and wallpaper. Query `?theme=light|dark` overrides on load and writes `localStorage`.

19.3 Verify dialog

Modal with your fingerprint words+hex and the selected peer's words+hex, plus the instruction to read them aloud. Escape or backdrop click closes it.

19.4 Message chrome

Incoming bubbles left, outgoing right. Group incoming shows the sender display name in the bubble. Time is locale `hh:mm`. Date chips: Today, Yesterday, or `toLocaleDateString()`. Signature badge and ticks sit in the bubble meta row.

19.5 Sidebar snippets

Last message preview: text as-is; normal image → 'Photo'; video → 'Video'; view-once → 'View once photo/video'; other files → name.

20. Relay server internals

`server.js` is a single Node process. State is two objects: `publicKeys` and `groups`. No database, no Redis, no TLS termination in-app (plain HTTP on :3000).

20.1 Static middleware

Paths `/`, `*.html`, `*.css`, `*.js` get `Cache-Control: no-store` so UI iteration is not stuck on cached JS. Then `express.static(public/)`.

20.2 Socket.IO options

`maxHttpBufferSize: 20 * 1024 * 1024`. Default `Engine.IO` limit is 1 MiB, which would reject encrypted videos. There is no custom adapter; rooms are in-process only (one Node instance).

20.3 relayToPeer

Helper used by send-message, typing, reactions, receipts, view-once-opened, and call-signal. Drops the event if `to` is missing, not a string, or equals the sender. Otherwise `io.to(to).emit(event, { from: socket.id, ...payload })`. It does not check that `to` is still in `publicKeys`; a disconnected target simply does not receive the packet.

20.4 Group send path

send-group-message is not `relayToPeer`. It checks `isGroupMember`, then `socket.to(groupId).emit('receive-group-message', { groupId, from, ... })`. Non-members cannot inject into the room via this handler. The server does not inspect ciphertext.

20.5 Sender-key distribution guard

Requires groupId, keyId, an array of distributions, and membership of the sender. Each element needs to, ciphertext, iv, signature; to $\neq$ sender; to must be a member. Valid rows are forwarded as group - sender - key.

21. Complete Socket.IO event catalog

Unless noted, payloads are JSON. Binary is not used; ciphertext is base64 inside JSON.

Event	Dir	Content on the wire	Encrypted?
register-key	C to S	encryptKey JWK, signKey JWK, displayName	No (public)
update-name	C to S	displayName string	No
peer-list	S to all	[{ id, displayName, encryptKey, signKey }]	No
send-message	C to S	to, msgType, ciphertext, iv, attachmentMeta?, keys, signature, msgId	Body yes
receive-message	S to peer	from + same fields	Body yes
msg-delivered / msg-read	relay	to, msgId	No
typing-start / typing-stop	relay	to	No
message-reaction	relay	to, msgId, emoji, action	No
view-once-opened	relay	to, msgId	No
call-signal	relay	to, data.{type, sdp?, candidate?, callType?}	No (SDP)
request-groups	C to S	(none)	-
group-list	S to clients	[{ id, name, members, memberCount }]	No
create-group	C to S	name	No
group-selected	S to creator	groupId	No
join-group / leave-group	C to S	groupId	No
distribute-group-sender-key	C to S	groupId, keyId, public JWKs, distributions[]	Key wrap yes
group-sender-key	S to member	from, groupId, keyId, ciphertext, iv, signature, JWKs	Key wrap yes
send-group-message	C to S	groupId, keyId, msgType, ciphertext, iv, meta, signature, msgId	Body yes
receive-group-message	S to room	groupId, from, + same (not echoed to sender)	Body yes
connect / disconnect	engine	socket id	-

Table 5. Every application event. Dir 'relay' means client to server to target peer. 'Body yes' means plaintext is not on the wire; ciphertext is.

22. In-memory client and server state

22.1 Server

- `publicKeys[socketId] = { encryptKey, signKey, displayName }`
- `groups[groupId] = { name, members: Set }`
- Socket.IO rooms named by groupId

22.2 Client maps and sets

Name	Holds
peerKeys / peerSigningKeys	Imported CryptoKey objects keyed by socket id
peerProfiles	displayName per socket id
visiblePeerIds	Online peers to render
typingPeers	Set of socket ids currently typing
groups	Map of group id → last group-list row
groupSenderKeyState	Per-group sender-key machine (§15.3)
conversations	Map of conversation key → { messages, unread, lastMessage }
drafts	Unsent composer text per conversation
messageRegistry	messageKey → message object (reactions)
messageStates	msgId → outgoing 1:1 message (ticks)
callSession	Single { peerId, callType, role, pc, localStream, pendingIce, timers } or null

23. Constants, limits, and identifiers

Constant	Value	Role
Port	3000	HTTP + Socket.IO
maxHttpBufferSize	20 MiB	Socket.IO incoming packet cap
ATTACHMENT_SIZE_LIMIT	12 MiB	Client reject before encrypt
AES key	256-bit	GCM
IV	12 bytes	Per encrypt
GCM tag	128-bit (Web Crypto default)	Appended to ciphertext
Curve	P-256	ECDH and ECDSA
Hash	SHA-256	ECDSA and fingerprints
Display name	32 chars	Sanitize slice
Typing idle	1200 ms	Auto typing-stop
Call ring/answer	40 s	Reject or 'No answer'
Toast	4200 ms	Auto-remove
Ringtone	680 Hz, 180 ms, every 1100 ms	Incoming call
STUN	stun:stun.l.google.com:19302	ICE only
Video ideal	1280×720, user camera	getUserMedia
Quick reactions	heart, thumbs-up, joy, fire	Picker (Unicode emoji in app.js)
Theme storage	e2ee.theme	light dark
Name storage	e2ee.displayName	string
Base64 chunk	0x8000 bytes	Avoid call-stack overflow

Table 6. Numbers you can grep in the source.

23.1 Identifier formats

- **socket.id** — assigned by Socket.IO (e.g. k-NNfG9x...). Routing key. Hidden in UI.
- **msgId** — `Date.now().toString(36) + Math.random().toString(36).slice(2,6)`. Unique enough for a session, not a UUID.
- **groupId** — `grp_` + same timestamp/random pattern.
- **sender keyId** — same generator as msgId.
- **conversationKey** — `peer:<id>` or `group:<id>`.
- **messageKey** — `sent:<msgId>`, `received:<peerId>:<msgId>`, or `group:<groupId>:<senderId>:<msgId>`.

24. Encoding and serialization

24.1 JWK public keys

`Web Crypto exportKey('jwk')` for EC P-256 public keys yields `{ kty:'EC', crv:'P-256', x, y, ext:true }` (and `key_ops`). These JSON objects are what the relay stores and fans out. Private JWKs are never exported in this codebase.

24.2 Base64 of binary

`bytesToBase64` walks the `Uint8Array` in 32768-byte windows, `String.fromCharCode(...subarray)` each window, concatenates, then `btoa`. Decode: `atob` then `charCodeAt` into a `Uint8Array`. This is standard base64, not base64url. IVs, ciphertexts, signatures, and wrapped sender keys all use it.

24.3 Text

UTF-8 via `TextEncoder` / `TextDecoder`. `JSON.stringify` for signature payloads with a fixed key order as written in source.

24.4 SDP

`RTCSessionDescription` objects are sent as the browser produces them (type + sdp string). ICE candidates are the `RTCIceCandidate` init dictionaries. No trickle policy beyond 'send every onicecandidate'.

25. Security properties and honest limitations

25.1 Properties you can claim for an honest, unmodified client

- The Node relay cannot read message text or attachment bytes.
- GCM tags detect bit-flips on ciphertext (with the usual AES-GCM caveats).
- ECDSA detects forged or altered signed payloads if the signing key is the one you fingerprinted alongside the encryption key.
- Group sender keys are wrapped to members pairwise; non-members are not sent wraps (server-enforced membership check plus client skip).
- WebRTC media is DTLS-SRTP between browsers when ICE succeeds.

25.2 Properties you must not claim

- **No forward secrecy.** One ECDH private key decrypts all 1:1 messages of that tab session.
- **No post-compromise security / ratchet.**
- **No persistent identity.** Refresh = new keys. You cannot verify a person across days except by reading fingerprints every session.
- **No metadata privacy.** Who talked to whom, when, sizes, names, SDP IPs are visible to the relay.
- **Display names are not authenticated.**
- **View once is not screenshot-proof and not cryptographically enforced.**
- **No TURN.** Calls can fail on hard NAT.
- **No certificate pinning of the web origin.** A malicious host serving this JS can steal keys.
- **No multi-device.** Each tab is a separate person.
- **No offline delivery.** If the peer is gone, the ciphertext is dropped.
- **Receipts and typing are forgeable** by anyone who can emit as that socket (i.e. the client or a compromised tab).
- **Call signaling is not bound to chat fingerprints** (no SAS on DTLS certs).
- **P-256 is not the Signal curve.** Implementation quality equals the browser's Web Crypto.
- **Single-process relay.** Restart wipes online users and groups.

25.3 XSS / injection

Message text is assigned with `textContent`, not `innerHTML`. Display names likewise. SVG icons are constants in source. This is the correct DOM discipline for not executing peer-controlled HTML.

26. Repository map

Path	Role
package.json	name e2ee, start script, dependencies express + socket.io
server.js	HTTP static + Socket.IO relay, all server events
public/index.html	DOM: sidebar, chat, modals, call overlays
public/styles.css	Light/dark tokens, layout, bubbles, call UI
public/app.js	All client crypto, UI, WebRTC, notifications
public/socket.io/socket.io.js	Served by the Socket.IO library, not a file in git

No test suite beyond node --check server.js. No TypeScript. No bundler.

27. Worked examples

27.1 Alice sends the text “Hello” to Bob

- Alice's tab already has ECDH (dA, QA) and ECDSA (sA, SA). Bob is in peer-list with QB, SB. Alice selected Bob.
- $\text{sharedKey} = \text{ECDH}(\text{dA}, \text{QB}) \rightarrow \text{AES-256-GCM key } K$. Bob will compute $\text{ECDH}(\text{dB}, \text{QA}) = K$.

- `iv = 12 random bytes. C = AES-GCM_K(UTF-8('Hello'), iv).`
- `sig = ECDSA-SHA256_sA( JSON {msgType:'text', text:'Hello'} ).`
- Alice → server: `send-message { to: bobSocket, msgType:'text', ciphertext:b64(C), iv:b64(iv), senderPublicKey:QA, senderSigningKey:SA, signature:b64(sig), msgId }.`
- Server → Bob: `receive-message { from: aliceSocket, ...same... }.`
- Bob emits `msg-delivered` immediately. Decrypts with K. Verifies sig with SB. Shows bubble + signed. Emits `msg-read`.
- Alice's ticks go ✓ then ✓✓ gray then ✓✓ blue. Relay saw sockets, sizes, and base64, not 'Hello'.

27.2 Alice starts a video call to Bob

- `getUserMedia(audio + 1280×720 user camera).` `RTCPeerConnection` with Google STUN. `addTrack` all tracks.
- `createOffer/setLocalDescription.` ICE gathering starts. Each candidate → `call-signal { type:'ice' }.`
- `call-signal { type:'offer', sdp, callType:'video' }` to Bob.
- Bob's tab is idle: incoming overlay, 680 Hz ringtone, 40 s timer. Bob clicks Accept (user gesture).
- Bob `getUserMedia`, new PC, `setRemoteDescription(offer)`, add queued ICE, `createAnswer`, `setLocalDescription`, signal answer.
- Alice `setRemoteDescription(answer)`, flush ICE. DTLS handshake, SRTP keys, `connectionstate connected`, timer 00:00.
- RTP audio/video flows peer-to-peer. Node only saw SDP and ICE (including candidates/IPs).
- Hang up: `stop tracks`, `pc.close()`, `call-signal hangup`.

27.3 View-once photo

- Alice picks a JPEG, toggles View once, Send. Bytes AES-GCM encrypted with K. `attachmentMeta.viewOnce=true` is inside the signed JSON.
- Bob decrypts, keeps bytes in RAM, shows a tile with no thumbnail.
- Bob taps: blob URL, fullscreen image. Close → revoke URL, bytes = null, tile 'Opened', view-once-opened to Alice.
- A dishonest Bob could have copied bytes before closing. The protocol cannot stop that.

28. Glossary

Term	Meaning in this project
Relay / server	The Node process. Not a CA, not an IdP, not a media SFU.
Peer	Another browser tab that registered keys. Identified by <code>socket.id</code> internally.
Pairwise key	AES-256-GCM derived via ECDH P-256 between two users.
Sender key	Per-member AES-256-GCM key for encrypting that member's group messages.
Fingerprint / safety number	Four words + 16 hex chars from SHA-256(raw ECDH public key).
Signed	ECDSA-P-256-SHA-256 over a canonical JSON payload verified.
View once	Honest-client flag to show media once, then drop RAM copies.

Term	Meaning in this project
STUN	Session Traversal Utilities for NAT — discovers public ICE candidates. Not a media relay.
TURN	Relays media when P2P ICE fails. Not deployed here.
SDP	Session Description Protocol — offer/answer text WebRTC uses to agree codecs and ICE.
ICE	Interactive Connectivity Establishment — candidate pairs until one path works.
DTLS-SRTP	How browsers encrypt WebRTC media after the DTLS handshake.
JWK	JSON Web Key — the export format of public EC keys on the wire.
GCM	Galois/Counter Mode — AEAD mode of AES used for all chat payloads.

End of specification. If a behavior is not in this document, it is not in the current codebase. Adding accounts, the Signal protocol, TURN, or persistent keys would be new work, not a hidden mode of what ships today.